

# BukScan - Wykorzystanie sztucznej inteligencji w projekcie

Przegląd narzędzi AI, modeli językowych i generatywnych użytych podczas budowy aplikacji | Czerwiec 2026

---

Projekt BukScan powstawał z dużym wykorzystaniem narzędzi AI na różnych etapach pracy - od planowania architektury i pisania kodu, przez generowanie treści i grafik, aż po sprawdzanie logiki biznesowej. W tym dokumencie opisujemy, jakich modeli i narzędzi używaliśmy, do jakich zadań oraz dlaczego do konkretnego problemu dobieraliśmy dany model lub poziom reasoning effort. Wybór nie był przypadkowy - zależał od trudności zadania i tego, ile tokenów dane zadanie zużyje, a nie od ślepego wybierania najmocniejszego modelu do wszystkiego.

## Narzędzia AI użyte w projekcie

---

### Claude Code (Pro)

*Główne narzędzie deweloperskie - praca w terminalu i edytorze*

To było nasze podstawowe środowisko pracy przez większość projektu. Claude Code działał jak agent bezpośrednio w terminalu - sam czytał pliki, edytował kod, uruchamiał komendy, czytał błędy i proponował poprawki, bez ręcznego kopiowania kodu w obie strony. W Claude Code korzystaliśmy z modeli Claude Opus 4.8, Opus 4.7 oraz Sonnet 4.6, dobierając je do trudności zadania.

- Projektowanie i implementacja architektury Next.js (App Router, SSR)
- Pisanie scraperów Playwright dla 10 bukmacherów
- Budowa silnika arbitrażu z obsługą podatku (src/lib/arbitrage.ts)
- Integracja Supabase (autentykacja, RLS, journal\_entries)
- Debugowanie błędów parsowania DOM i normalizacji nazw drużyn
- Refaktoryzacja i optymalizacja komponentów React/TypeScript

### GitHub Copilot (VS Code)

*Autouzupełnianie i chat inline w edytorze*

Copilota używaliśmy równolegle z Claude Code, głównie w VS Code do szybkiego autouzupełniania i drobnych poprawek inline. Przydawał się zwłaszcza przy powtarzalnych fragmentach kodu - typach TypeScript, komponentach Tailwind, handlerach API. W Copilocie korzystaliśmy z różnych modeli w zależności od potrzeby: GPT 5.4 Codex, GPT 5.5, a także modeli Claude (Opus 4.8, Opus 4.7, Sonnet 4.6).

- Autouzupełnianie typów TypeScript i interfejsów
- Generowanie boilerplate komponentów React
- Szybkie poprawki w CSS/Tailwind (klasy, responsywne breakpointsy)
- Uzupełnianie powtarzalnych wzorców (np. obsługa błędów w API routes)
- Chat inline do tłumaczenia fragmentów kodu bez zmiany kontekstu

## Gemini (Google)

*Koncepcja, research, weryfikacja pomysłów*

Gemini wykorzystywaliśmy głównie na etapie planowania i koncepcji - kiedy potrzebowaliśmy szybko sprawdzić pomysł, porównać podejścia albo zrobić research na temat algorytmów bez wciągania w to całego kontekstu projektu. Używaliśmy modeli Gemini 3.1 Pro (do trudniejszych pytań) oraz Gemini 3.5 Flash (do szybkich, prostszych zapytań).

- Research: mechanizmy antybotowe polskich bukmacherów
- Porównanie podejść do normalizacji nazw drużyn (tokenowe vs. Levenshtein)
- Koncepcja algorytmu filtrowania watchlisty (near-arbitrage)
- Weryfikacja matematyki arbitrażu z uwzględnieniem podatku od obrotu
- Research: dostępność publicznych API polskich bukmacherów

## Nano Banana Pro (Google)

*Generowanie grafik - tła aplikacji*

Do stworzenia tła i grafik aplikacji użyliśmy generatora obrazów Nano Banana Pro od Google. Model dobrze radzi sobie z realistycznymi obrazami w wysokiej jakości i pozwala dość precyzyjnie sterować stylem przez prompt.

- Generowanie ciemnych, abstrakcyjnych tła aplikacji (dark-themed gradients)
- Tworzenie grafik hero section - stadion, piłka, efekty świetlne
- Obrazy dla trybu demo - wizualizacje bukmacherów i kursów
- Eksport w formatach WebP (optymalizacja wydajności) i PNG

## Dobór modeli i poziomu reasoning effort do zadań

---

| Model                                             | Effort       | Typ zadań                                               | Przykłady użycia w projekcie                                                                                                                                                                                                                     |
|---------------------------------------------------|--------------|---------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Claude Opus 4.8                                   | HIGH / MAX   | Najtrudniejsze zadania architektoniczne i algorytmiczne | Projektowanie silnika arbitrażu z obsługą podatku; algorytm scalania wydarzeń między bukmacherami (normalizacja tokenowa ≥65%); debugowanie złożonych błędów scrapera Playwright; decyzje architektoniczne Next.js App Router                    |
| Claude Opus 4.7                                   | HIGH         | Złożone zadania wymagające głębszego rozumowania        | Weryfikacja matematyki surebetu po podatku, analiza edge-case'ów w normalizacji nazw drużyn (np. "DR Kongo" w różnych zapisach), planowanie schematu Supabase z RLS                                                                              |
| Claude Sonnet 4.6                                 | MEDIUM       | Standardowe zadania implementacyjne - dominujący model  | Pisanie komponentów React/TypeScript, konfiguracja Tailwind CSS, implementacja endpointów Next.js API, integracja Supabase Auth (Google OAuth), budowa widoków /events, /margins, /journal, /calculator, refaktoryzacja istniejących komponentów |
| Claude Sonnet 4.6                                 | LOW          | Proste, powtarzalne zadania - szybkie odpowiedzi        | Poprawki stylistyczne CSS/Tailwind, zmiana nazw zmiennych, dodawanie typów TypeScript do istniejących funkcji, proste formatowanie danych, drobne poprawki UX, generowanie dokumentacji inline                                                   |
| GitHub Copilot (GPT 5.4 Codex / GPT 5.5 / Claude) | LOW          | Autouzupełnianie inline, chat w VS Code                 | Uzupełnianie powtarzalnych wzorców kodu, autouzupełnianie typów TS, boilerplate komponentów, szybkie pytania o API bez przełączania kontekstu z edytora                                                                                          |
| Gemini 3.1 Pro / 3.5 Flash                        | MEDIUM / LOW | Research, koncepcje, weryfikacja niezależna             | Research techniczny (antyboty, scraping polskich bukmacherów), porównanie algorytmów normalizacji, weryfikacja matematyki arbitrażu niezależna od Claude, brainstorming funkcji produktu. 3.1 Pro do trudniejszych pytań, 3.5 Flash do szybkich  |
| Nano Banana Pro                                   | -            | Generowanie obrazów, zasoby wizualne                    | Tła aplikacji w stylu dark UI, grafiki hero, zdjęcia dla trybu demo. Promptowanie w języku angielskim z opisem stylu (dark, cinematic, stadium lights, abstract bokeh)                                                                           |

## Typowy workflow deweloperski z AI

Poniżej opisujemy, jak w praktyce wyglądała współpraca człowiek-AI przy budowie BukScan, krok po kroku.

### 1. Koncepcja i research (Gemini 3.1 Pro / 3.5 Flash)

Research: jak działają antyboty polskich bukmacherów, jakie podejścia do normalizacji nazw drużyn sprawdzają się w praktyce, jak liczy się arbitraż z podatkiem od obrotu. Gemini służył też jako "drugi głos" do niezależnej weryfikacji matematyki przed implementacją. Trudniejsze pytania szły do 3.1 Pro, szybkie do 3.5 Flash.

### 2. Projektowanie architektury - najtrudniejsze decyzje (Claude Opus 4.8 / 4.7, effort HIGH)

Decyzje architektoniczne: struktura scraperów (generyczny ekstraktor vs. dedykowane strategie), schemat danych w Supabase (RLS, `journal_entries`), logika silnika arbitrażu z obsługą wyjątków podatkowych. Tu trafił też problem stabilnych ID wydarzeń niezależnych od scrapingu. Te zadania były na tyle złożone i czasochłonne, że opłacało się przeznaczyć na nie więcej tokenów i wyższy effort.

### 3. Implementacja głównych funkcji (Claude Sonnet 4.6, effort MEDIUM, w Claude Code)

Pisanie komponentów React, endpointów API, integracja Supabase Auth, budowa widoków. Claude Code działał jako agent - sam czytał pliki projektu, edytował kod, uruchamiał testy i interpretował błędy. To było dominujące środowisko przez cały projekt.

### 4. Praca równoległa w VS Code (GitHub Copilot - GPT 5.4 Codex / GPT 5.5 / Claude)

Przy edycji plików bezpośrednio w VS Code korzystaliśmy z Copilota do szybkich poprawek Tailwind, typów TypeScript i boilerplate. Chat Copilota inline służył do pytań o konkretne fragmenty bez wychodzenia z edytora. Model dobieraliśmy zależnie od tego, czy chodziło o proste uzupełnienie, czy o coś bardziej wymagającego.

### 5. Drobne poprawki i refaktoryzacja (Claude Sonnet 4.6, effort LOW)

Poprawki stylistyczne, zmiana nazw, formatowanie, drobne zmiany UX. Effort "low" wyraźnie przyspiesza odpowiedź przy prostych zadaniach - używanie wysokiego effort do zmian w CSS byłoby marnowaniem czasu i tokenów.

### 6. Debugowanie złożonych błędów (Claude Opus 4.8, effort HIGH/MAX)

Najtrudniejsze błędy (parsowania DOM, nieprawidłowe scalanie meczów o tej samej nazwie, ale różnym czasie, edge-case'y podatkowe) kierowaliśmy do Opus z wysokim effortem. Model z większym budżetem reasoning lepiej radził sobie z wieloetapowym debugowaniem.

### 7. Generowanie zasobów wizualnych (Nano Banana Pro)

Tła i grafiki aplikacji generowane przez Nano Banana Pro. Prompty opisowe po angielsku: dark UI aesthetic, stadium atmosphere, bokeh lights, cinematic color grading. Eksport do WebP dla szybszego ładowania strony.

## Zasada doboru modelu do trudności zadania

**Najważniejsza obserwacja:** nie każde zadanie wymaga najmocniejszego modelu. Używanie Claude Opus z effort MAX do prostej zmiany koloru przycisku jest jak używanie młota do wbicia pinezki - droższe (więcej tokenów), wolniejsze i zwyczajnie niepotrzebne. Dlatego model i poziom effort dobieraliśmy świadomie, pod trudność zadania i jego koszt w tokenach, a nie wybieraliśmy najmocniejszego modelu z automatu.

W praktyce wyglądało to tak:

- **Opus 4.8 / 4.7 (effort HIGH/MAX)** - decyzje architektoniczne, algorytmy, najtrudniejsze błędy. Model "myśli" dłużej i głębiej, co ma sens przy wieloetapowych problemach logicznych, mimo wyższego kosztu w tokenach.

- **Sonnet 4.6 (effort MEDIUM)** - standardowa implementacja, komponenty, integracje. Dobry stosunek jakości do prędkości - to był model używany przez większość czasu projektu.
- **Sonnet 4.6 (effort LOW)** - proste poprawki, CSS, typy. Odpowiedź prawie natychmiastowa, a jakość w zupełności wystarczająca.
- **Copilot inline (GPT 5.4 Codex / GPT 5.5 / Claude)** - autouzupełnianie bez przerywania pracy w edytorze.
- **Gemini 3.1 Pro / 3.5 Flash** - niezależna weryfikacja i research, bez "zaślepienia" kontekstem projektu z Claude.

## Wnioski z używania AI w projekcie

---

### Co AI zmieniło w procesie budowy projektu

- **Prędkość prototypowania:** pierwsze działające widoki powstawały w godzinach, nie w dniach. AI generowało boilerplate i strukturę, dzięki czemu mogliśmy skupić się na logice biznesowej.
- **Mniej błędów:** Claude Code przy edycji kodu od razu wskazywał błędy typów TypeScript i niezgodności z istniejącym kodem, bez uruchamiania pełnego buildu.
- **Niezależna weryfikacja:** używanie Gemini jako "drugiego głosu" do sprawdzenia matematyki arbitrażu zmniejszyło ryzyko błędu logicznego w kluczowym silniku aplikacji.
- **Generowanie grafik:** Nano Banana Pro pozwolił uniknąć kupowania stock photo albo angażowania grafika - tła powstawały w kilka minut od promptu.
- **Dobór narzędzia do zadania:** mieszanie modeli (Opus do trudnych, Sonnet do rutynowych, Copilot do inline) okazało się skuteczniejsze niż używanie jednego modelu do wszystkiego, również pod kątem kosztu w tokenach.
- **Ograniczenia:** AI nie zastąpiło wiedzy domenowej - zrozumienie mechaniki surebetów, polskiego prawa podatkowego (12% od obrotu) i zachowania scraperów wymagało własnej analizy i decyzji projektowych.